

КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

01

Лекция 1
Введение в C++. Инструменты разработки

История C++ и стандарты · структура программы · трансляция и ODR · make и CMake · Google Test · отладка · типы и инициализация

МАИ · лекции курса ООП

C++20 | GCC 14 | 12 глав

О себе



Дзюба Дмитрий Владимирович
старший преподаватель ОЦ №8
руководитель центра разработки архитектурных систем
ООО Вымпелком-ИТ

- 20 + лет опыта в разработке ИТ систем в области телекоммуникаций, взаимоотношений с клиентами, геоинформационных систем, аналитических систем …
- опыт выстраивания архитектурного процесса в Билайн и компаниях экосистемы МТС

Хобби: c++ :-)

Контакт: ddzuba@Yandex.ru



Сроки, экзамен, правила работы

Лабораторные работы и сроки

В семестре **6 лабораторных работ**.

На защиту каждой — **две недели**.

Экзамен

На экзамене нужно **писать код руками**.

Программистом становятся, когда пишут программы.

Правила работы

«Вайб-кодить» запрещено.

Код — лучший способ обучения абстрактному мышлению.

Парадигмы

процедурная · ООП · функциональная — C++ мультипарадигменный

Инструменты

VS Code · **GCC 14** · make/CMake · gtest · gdb/lldb

> Относитесь к лабораторным работам как к **абонементу в тренажёрный зал**.



Бонус

В середине семестра будет контрольная, где будем писать код руками (благодарим Cloudcode и Codex)

Тренируем образ мышления

A = B



A тождественно B

B копируется в A

Календарь семестра

нед.1 08.09	нед.2 15.09	нед.3 22.09	нед.4 29.09	нед.5 06.10	нед.6 13.10	нед.7 20.10	нед.8 27.10	нед.9 03.11	нед.10 10.11	нед.11 17.11	нед.12 24.11	нед.13 01.12	нед.14 08.12	нед.15 15.12	нед.16 22.12
Л1 Введение	Л2 Операторы	Л3 Указатели	Л4 Функции	Л5 Классы	Л6 Конструкт.	Л7 Copy/Move	Л8 Перегрузка	Л9 Наследование	Л10 Полиморфизм	Л11 Множ. насл.	Л12 Исключения	Л13 Умн. указ.	Л14 SOLID	Л15 STL	Л16 POCO
				ЛР 1		ЛР 2		ЛР 3		ЛР 4		ЛР 5		ЛР 6	

группа 201 четверг нижняя неделя (10/09 – 17/12) 16:30 – 19:45 — Дзюба Д

группа 202 четверг верхняя неделя (17/09 – 24/12) 16:30 – 19:45 — Крылов СС

группа 203 четверг нижняя неделя (17/09 – 24/12) 09:00 – 12:15 — Сысоев М

группа 204 четверг верхняя неделя (17/09 – 24/12) 13:00 – 16:15 — Ирбитский И

группа 205 суббота нижняя неделя (12/09 – 17/12) 16:30 – 19:45 — Ирбитский И

группа 206 пятница верхняя неделя (18/09 – 17/12) 9:00 – 12:15 — Ядров А

- считаем балы за ЛР
- до 15 балов при условии сдачи вовремя
- до 10 балов в случае если нет вовремя

М1 Основы языка

М2 Классы и ресурсы

М3 Наследование и полиморфизм

М4 Проектирование + STL + POCO



ПЛАН ЛЕКЦИИ

План занятия

01 История C++ · C vs C++

02 Стандарты C++11/14/17/20

03 Структура программы · main

04 Этапы трансляции

05 Правило одного определения

06 Система сборки make

07 Система сборки CMake

08 Введение в Google Test

09 Отладка gdb / lldb

10 Типы, литералы, sizeof

11 Инициализация переменных

12 const, constexpr, auto, namespace

Сквозная идея: путь от исходного кода до работающей и протестированной программы — препроцессор, компилятор, линковщик, сборка, тесты, отладка.

От «C with Classes» к современному C++

Аспект	C	C++
Парадигма	Процедурная	Мультипарадигменная
Типизация	Слабая	Более строгая
Память	malloc/free	new/delete, RAII
Библиотека	stdio, stdlib	STL
Полиморфизм	указатели на функции	виртуальные функции, шаблоны

- Создан Бьёрном Страуструпом в Bell Labs, начало 1980-х
- идея — добавить к C **абстракции данных и ООП**
- Rust: те же задачи, но строже гарантии и выше порог входа

Роль C++ в курсе

Скорость C + гибкость Python. C++ учит **декомпозировать** сложные задачи на простые и строить модель программы, которую легко расширять и менять.

Обновления каждые три года

C++11 «современный C++»: auto, лямбды, nullptr, constexpr, умные указатели

C++14 шлифовка: обобщённые лямбды, relaxed constexpr

C++17 structured bindings, if constexpr, std::optional, std::string_view

C++20 concepts, ranges, coroutines, modules, std::format, <=>

C++23… продолжение развития языка

```
g++ -std=c++20 -Wall -Wextra \
    -o program main.cpp
```

В этом курсе

работаем со стандартом **C++20** (GCC 13+ / GCC 14).

Точка входа — main

```cpp
// main.cpp
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << std::endl;
    return 0;
}
```

- стандарт допускает две формы:

```cpp
int main() { ... }
int main(int argc, char* argv[]) { ... }
```

- argc — число аргументов командной строки
- argv — массив C-строк с аргументами
- return 0 — успех (можно опустить)

ПРИМЕР

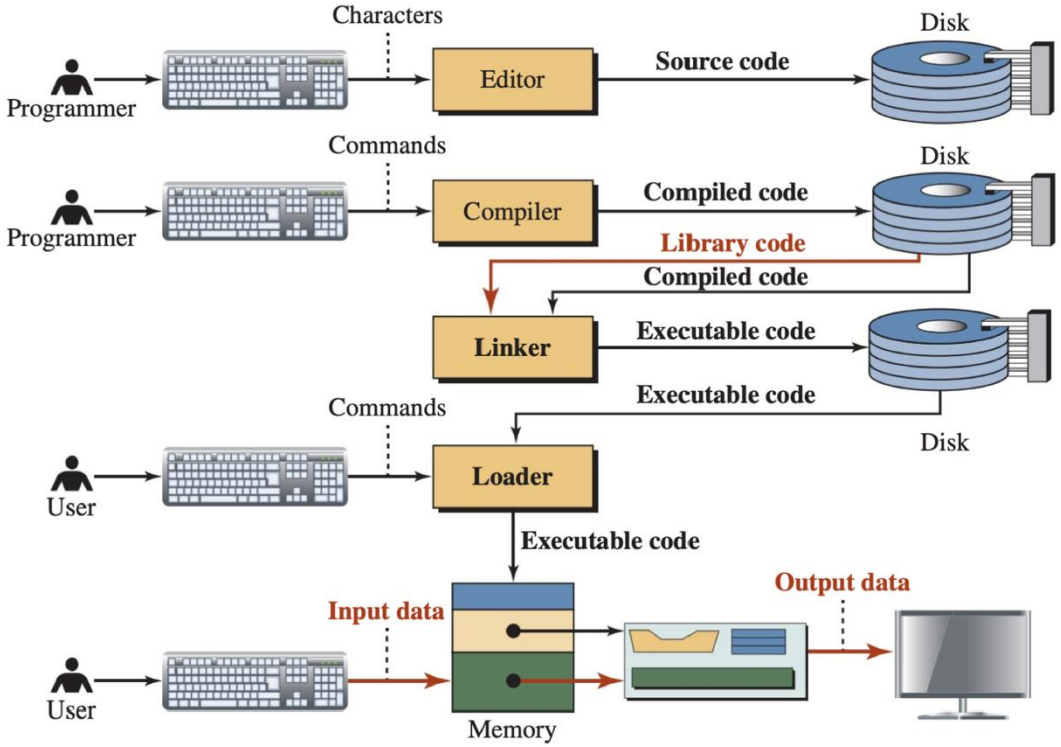
Смотрим пример: 01_FirstCppProgram

Разбираем после теории

Первая программа · точка входа main

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

Препроцессор → компилятор → линковщик



- **Препроцессор** — текстовый процессор; результат — source.i

```
g++ -E main.cpp -o main.i
```

- **Компилятор** — каждая единица трансляции независимо → .o

```
g++ -c main.cpp -o main.o
```

- **Линковщик** — собирает объектные файлы, разрешает символы

```
g++ main.o utils.o -o program
```

ПРИМЕР

Смотрим пример: 02_translation_units

Разбираем после теории

Этапы трансляции

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

```
// Пример 02: единицы трансляции и ODR.
//
// Программа собирается из двух единиц трансляции:
// main.cpp — использует функции;
// math_utils.cpp — определяет add().
// Заголовок math_utils.h только ОБЪЯВЛЯЕТ add() и определяет inline square().
//
// Сборка вручную (что под капотом делает CMake):
// g++ -std=c++20 -c math_utils.cpp -o math_utils.o
// g++ -std=c++20 -c main.cpp -o main.o
// g++ math_utils.o main.o -o program # линковщик связывает символы
```

Правило одного определения

```
// utils.h — объявление
int add(int a, int b);

// utils.cpp — определение
int add(int a, int b) {
        return a + b;
}

// Защита заголовка от повторного включения
#pragma once                    // современный способ

// или include guard: #ifndef
UTILS_H #define UTILS_H
// содержимое заголовка
#endif
```

- В единице трансляции — не более одного определения
- Во всей программе функция/переменная с внешней связью — ровно одно определение
- Классы, шаблоны, inline— в нескольких единицах, но определения **идентичны**
- нарушение ODR — **undefined behavior**

Разделение

в .h— объявления, в .cpp— определения.

Декларативная сборка по правилам

```
# Правило (rule):
#       цель: зависимости #
        <табуляция>команда

main.o: main.cpp functions.h
                g++ -c main.cpp -o main.o

program: main.o utils.o
                g++ main.o utils.o -o program
```

Критично: отступ в команде — символ **табуляции**, пробелы не допускаются.

- **Инкрементальная сборка:** пересобирается только то, что изменилось (по временным меткам)
- **Декларативный подход:** описываем *что* собрать, а не *как*
- make сам строит **граф зависимостей** и порядок выполнения
- цель — файл, который нужно получить; зависимости — файлы, из которых он создаётся

Проект вместо ручного Makefile

```
cmake_minimum_required(VERSION 3.16)
project(MyFirstProject LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(hello main.cpp) #
add_library(...) —библиотека

target_compile_options(hello PRIVATE -Wall -Wextra -Wpedantic)
```

```
mkdir build && cd build
cmake ..                  # генерация файлов сборки #
cmake --build .           компиляция
./hello                   # запуск
```

Режим	Флаги
Debug	-g -O0
Release	-O3 -DNDEBUG
RelWithDebInfo	-O2 -g -DNDEBUG

> Без CMAKE_BUILD_TYPECMake собирает **без -g**— отладчик не найдёт точку останова. Проекты курса — **Debug по умолчанию**.

Полное описание

HTTPS://CMAKE.ORG/CMAKE/HELP/LATEST/GUIDE/TUTORIAL/INDEX.HTML



Модульное тестирование кода

```
include(FetchContent)
FetchContent_Declare(
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git GIT_TAG v1.14.0
)
FetchContent_MakeAvailable(googletest)

add_executable(tests tests.cpp) target_link_libraries(tests
GTest::gtest_main)
```

```
#include <gtest/gtest.h>

int add(int a, int b) { return a + b; } TEST(AddTest,

PositiveNumbers) {
        EXPECT_EQ(add(2, 3), 5);
        EXPECT_EQ(add(0, 0), 0);
}

TEST(AddTest, NegativeNumbers) {
        EXPECT_EQ(add(-1, -1), -2);
        EXPECT_EQ(add(-1, 1), 0);
}
```

> Запуск: cd build && ctest --output-on-failure.

ПРИМЕР

Смотрим пример: 03_gtest_calculator

Разбираем после теории

Калькулятор с тестами

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

```
include(FetchContent)
FetchContent_Declare(googletest
GIT_REPOSITORY https://github.com/google/googletest.git
GIT_TAG v1.15.2)
FetchContent_MakeAvailable(googletest)
enable_testing()
include(GoogleTest)

...

target_link_libraries(my_test PRIVATE GTest::gtest_main)
```

Точки останова, шаги, стек вызовов

Команда	Кратко	Назначение
break main.cpp:12	b	точка останова
run	r	запуск
next	n	шаг без захода
step	s	шаг с заходом
print x	p	значение переменной
backtrace	bt	стек вызовов
continue	c	продолжить
quit	q	выход

```
g++ -g -O0 -o program main.cpp
$ lldb ./program (lldb) b
main.cpp:14 (lldb) run
(lldb) next / step (lldb)
print argc (lldb) bt
(lldb) continue
```

> break main ненадёжен (перегрузки, namespace) — ставьте по **файлу:строке**.

Целые числа и память на стеке

```
short s = 32767;                          // минимум 16 бит
int i = 2'147'483'647;                    // обычно 32
long long ll = 9'223'372'036'854'775'807LL;

// Типы фиксированного размера — рекомендуются int8_t
                i8      = 127;
int32_t i32 = 2'147'483'647;
int64_t i64 = 9'223'372'036'854'775'807LL;

sizeof(int)               // 4
int million = 1'000'000;                  // разделитель (C++14)
int dec = 42, oct = 052,                  // 0 — 8-ричная!
        hex = 0x2A, bin = 0b101010;
```

```
float f = 3.14f; double d =          // 4 байта
3.14159; bool flag = true;           // 8 байт
char c = 'A';                        // 1 байт
                                     // 1 байт

// составные операторы — справа налево
int x = 10, y = 20;
y += x *= 40;              // x=400, y=420
```

Стек

локальные переменные — на стеке: автоматическое время жизни, быстро, но **размер ограничен** (единицы МБ). Неинициализированная переменная — «мусор» (UB).

ПРИМЕР

Смотрим пример: 04_basic_types

Разбираем после теории

Базовые типы

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

```
std::int8_t i8 = 127;
std::int32_t i32 = 2'147'483'647; // разделитель разрядов ' (C++14)
std::int64_t i64 = 9'223'372'036'854'775'807LL;
```

Type	Sign	Range	
short int	signed	–32,768	+32,767
	unsigned	0	65,536
int	signed	–2,147,483,648	+2,147,483,647
	unsigned	0	4,294,967,295
long int	signed	–2,147,483,648	+2,147,483,647
	unsigned	0	4,294,967,295

Размеры типов

Literal	Description	Literal	Description
false	// Boolean literal	12897234L	// long integer literal
'A'	// character literal	245.78F	// float literal
"Hello"	// string literal	114.7892	// double literal
234	// integer literal	245.784321L	// long double literal

Литералы

Четыре формы и почему предпочтителен {}

```
int a = 42;          // copy initialization
int b(42);           // direct initialization
int c{42}; int d =   // list (C++11)
{42}; int e{};       // copy-list
int f = {};          // value — гарантированно 0
                     // тоже 0
```

- **Защита от сужения**: int x = 3.14;— тихое усечение; int y{3.14};— ошибка компиляции
- **Единообразие** для всех типов
- Защита от **most vexing parse**: Widget w();— объявление функции, Widget w{};— объект

> Предпочитайте **фигурную инициализацию** {}.

ПРИМЕР

Смотрим пример: 05_initialization

Разбираем после теории

Инициализация переменных

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

Неизменяемость, вывод типа, изоляция имён

```
const int max_size = 100;                  // неизменяемо
// max_size = 200;                         // ошибка

constexpr int square(int x) { return x * x; }constexpr int r = square(5); //
compile-time static_assert(r == 25);


auto x = 42;   auto y =          // int
3.14; auto f = &square;          // double
// auto a;                      // double (*)(double)
                                 // ошибка: нет инициализатора
```

	const	constexpr
Когда	может в рантайме	обязано compile-time
Контекст	любой	compile-time аргументы
static_assert	не всегда	да

```
namespace math        { double pi = 3.14159; }
namespace physics { double c = 299'792'458.0; }
using math::square;                // предпочтительно
// using namespace std;            // не в заголовках!
```

> auto в параметрах функций — неоднозначность; не злоупотребляйте там, где тип должен быть очевиден.

ПРИМЕР

Смотрим пример: 06_const_constexpr

Разбираем после теории

const / constexpr

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

ПРИМЕР

Смотрим пример: 07_auto

Разбираем после теории

auto

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

ПРИМЕР

Смотрим пример: 08_namespaces

Разбираем после теории

Пространства имён

- Полный код примера — в репозитории курса
- Запустите и разберите его, сверяясь с теорией
- Подсказки и комментарии — внутри примера

alignas	alignof	and	and_eq	asm
auto	bitand	bitor	bool	break
case	catch	char	char16_t	char32_t
class	compl	const	const_cast	constexpr
continue	decltype	default	delete	do
double	dynamic_cast	else	enum	explicit
export	extern	false	float	for
friend	goto	if	inline	int
long	mutable	namespace	new	noexcept
not	not_eq	nullptr	operator	or
or_eq	private	protected	public	register
reinterpret_cast	return	short	signed	sizeof
static	static_assert	static_cast	struct	switch
template	this	thread_local	throw	true
try	typedef	typeid	typename	union
unsigned	using	virtual	void	volatile
wchar_t	while	xor	xor_eq	

ключевые слова c++

ИТОГИ

Ключевые концепции лекции

- **C++ — мультипарадигменный**: процедурное, ООП, обобщённое, функциональное
- **Этапы сборки:** препроцессор → компилятор → линковщик
- **ODR** — одно определение; нарушение → UB
- **Инструментарий:** VS Code · GCC 14 · make/CMake · gdb/lldb · Google Test
- **Фигурная инициализация {}**защищает от сужения и most vexing parse
- const— неизменяемость, constexpr— compile-time
- autoвыводит тип из инициализатора — но не злоупотребляйте
- **Отладка:** -g -O0, точки останова, next/step, стек через bt
- **Стек:** автоматическое время жизни, ограниченный размер — осторожно с рекурсией и большими массивами

> Сборка, тесты и отладка — базовые навыки: без них невозможна ни одна лабораторная работа курса.

ПРАКТИКА

Что чаще всего идёт не так

```
// 1. using namespace std; в заголовке
#pragma once
using namespace std;                // ПЛОХО — грязнит глобально

// 2. #define вместо constexpr
#define MAX_SIZE 100               // ПЛОХО: нет типа
constexpr int max_size = 100;                  // ХОРОШО

// 3. Неинициализированная переменная
int x;          // мусор! (UB)
int y{}; // гарантированно 0
```

```
// 4. Путаница с системами счисления
int a = 010;              // это 8, а НЕ 10!

// 5. Большой массив на стеке
int data[1'000'000];                  // ~4 МБ → stack overflow

// 6. Отладка без -g
gdb ./program             // «no debugging symbols found»
// собирайте с -g -O0
```

> Забытый returnв mainдопустим, но для любой другой не-voidфункции — undefined behavior.

КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

01

Лабораторная работа № 1 Основы C++ — одна программа на динамических структурах данных

Чистые функции · указатели и ссылки · out-параметры · текстовое меню · CMake и Google Test

МАИ · материал лекций 1–4

C++17 | CMake | Google Test

ИДЕЯ

Одна программа, а не набор примеров

Вы напишете **одну законченную консольную программу** — интерактивный справочник, который работает с динамической структурой данных вашего варианта и выполняет операции из **текстового меню**.

✓

Часть 1 · Программа

Динамическая структура (массив, C-строка или матрица), базовые операции семейства, алгоритм варианта и поиск с **out-параметром**.

✓

Часть 2 · Тесты

CMake-проект с **Google Test**: чистые функции проверяются тестами, а не только работой в меню.

✓

Чистые функции

Функции не вводят и не выводят сами: **результат — через return, найденное значение — по ссылке**.

> Один проект: **чистые функции** вызываются из меню программы, а **тесты** проверяют их поведение на независимых данных.

ОГРАНИЧЕНИЯ

Что можно и чего нельзя

✓ Можно

- Чистые функции: параметр → результат
- Передача по значению, **ссылке**, **указателю**, out-параметры
- Ручная память: `new[]`/ `delete[]`
- С-строки `char*`с завершающим `'\0'`
- Арифметика указателей и `const`

✗ Нельзя

- Классы и конструкторы
- Семантика копирования / перемещения
- `std::string`, `std::vector`, `std::array`
- Умные указатели (`unique_ptr`и т. п.)

Почему? Пройдены только лекции 1–4. Работа — «руками»: почувствовать память, указатели и ссылки до того, как за них возьмутся классы и стандартные контейнеры.

КЛЮЧЕВАЯ ИДЕЯ

Поиск: результат — по return, значение — по ссылке

```
// Сигнатура out-параметра:
//       return       — true/false «нашли?»
//       out          — записываем найденное значение
bool find_max(const int* arr, size_t size,
                          int& out_value) {
        if (size == 0) return false; out_value =
        arr[0];
        for (size_t i = 1; i < size; ++i)
                if (arr[i] > out_value) out_value =
                        arr[i];
        return true;
}
```

```
int value = 0;
if (find_max(data, 4, value)) {std::cout <<
        "Максимум: "
                                << value << '\n';
} else {
        std::cout << "Массив пуст\n";
}
// → Максимум: 9
```

Снаружи

int v; if (f(arr, n, v)) { std::cout << v; }— результат проверяем, значение читаем из v.

Функция **не печатает сама**: вызывающий решает, что делать. Никаких «магических» значений вроде -1«если не нашли» — для этого есть boolи ссылка.

Общая схема: меню вокруг структуры данных

Пункты меню (общие)

1. Создать структуру данных
2. Напечатать
3. Операция семейства
4. Алгоритм варианта

0. Выход

- **Без утечек:** на каждый `new[]` — парный `delete[]`; после удаления — `arr = nullptr`
- **Корректный ввод:** `if (!(std::cin >> n)) { clear; continue; }`
- **Пустая структура:** поиск и алгоритмы корректны при длине 0
- **Каждая операция выводит понятный результат:** сам массив, длину, найденные значения

> Переменная структуры (указатель, `size`) объявляется **до цикла** и живёт между итерациями меню.

ВАРИАНТЫ

Вариант задаёт структуру данных

Семейство	Варианты	Структура данных	Файл операций
A · Линейный динамический массив	1 – 8	int*+ размер	array_ops
B · C-строки через указатели	9 – 14	char*+ '\0'	str_ops
C · Двумерные массивы, матрицы	15 – 20	int**— массив строк	matrix_ops

Базовые операции семейства — общие для всех его вариантов. **Алгоритм и дополнительные функции — только ваши**: подробная таблица вариантов и «до → после» каждого алгоритма — в задании.

Линейный динамический массив

```
// array_ops.h
int* array_create(std::size_t size);
void array_delete(int*& arr);
// resize/insert/remove — НОВАЯ память + копия,
// возвращают новый указатель; старый удалить!
int* array_resize(int* arr, std::size_t size,
                  std::size_t new_size);
int* array_insert(int* arr, std::size_t& size,
                  std::size_t pos, int value);
int* array_remove(int* arr, std::size_t& size,
                  std::size_t pos);
void array_print(const int* arr, std::size_t size);
```

Вставка/удаление: **новый массив** → скопировать → **удалить старый** → обновить указатель и размер.

Меню семейства А

1 Создать массив
2 Напечатать
3 Вставить элемент
4 Удалить элемент
5 Изменить размер
6 Печать
7 Алгоритм варианта
0 Выход

* Пример (варианты 1–4)

```
bool array_binary_search(
        const int* arr, std::size_t size,
        int target, std::size_t& out_index);
```

C-строки через указатели char*

```
// str_ops.h
char* str_alloc(const char* src);
        // копия строки в новой динамической памяти
std::size_t str_len(const char* s);
        // длина до '\0'
void str_copy(char* dst, const char* src);
        // копирование в существующий буфер
void str_delete(char*& s);
        // delete[] и обнулить
void str_print(const char* s);
```

Плюс один байт: под строку всегда выделяется strlen + 1байт — последний под завершающий '\0'. Иначе — «проезд» за границу.

Меню семейства В

1Ввести строку	2Напечатать
3Длина	4Скопировать в буфер и напечатать
5Алгоритм варианта	0Выход

* Пример (вариант 10)

```
std::size_t str_count_words(const char* s);
bool str_find_substr(const char* s,
        const char* sub, int& out_pos);
```

Двумерные массивы, матрицы int**

```
// matrix_ops.h
int** matrix_create(std::size_t rows,
                    std::size_t cols);
        // массив указателей на строки
void matrix_delete(int** m, std::size_t rows);
        // каждая строка, затем массив строк
void matrix_fill(int** m, std::size_t rows,
                 std::size_t cols, int value);
void matrix_print(const int* const* m,
                  std::size_t rows, std::size_t
                  cols);
```

Матрица — **массив указателей на строки**: m = new int*[rows], каждая строка — m[i] = new int[cols]. Удаление — в обратном порядке.

Меню семейства С

1 Создать матрицу
2 Заполнить значением
3 Напечатать
4 Алгоритм варианта
0 Выход

* Пример (вариант 15)

```
int** matrix_transpose(
        const int* const* m,
        std::size_t rows, std::size_t cols);
```

РЕЗУЛЬТАТ

Как выглядит работающая программа

```
$ cmake -B build && cmake --build build
$ ./build/lab_01_main
1.  Создать массив          5.  Изменить размер
2.  Напечатать              6.  Печать
3.  Вставить элемент        7.  Алгоритм варианта
4.  Удалить элемент         0. Выход
Ваш выбор: 3
Позиция: 2        Значение: 9
Массив: { 3, 9, 4, 1 }              →    { 3, 9, 9, 4, 1 }

Ваш выбор: 6
Массив: { 3, 9, 9, 4, 1 }

$ ctest --test-dir build
[==========] Running 10 tests from 2 test suites.
[     PASSED     ] 10 tests.
```

Каждое действие — вызов **чистой функции**. Программа сама управляет памятью и не падает на неверном вводе.

Структура проекта и CMake

```
lab_01/
 ├── CMakeLists.txt
 ├── main.cpp                    # программа варианта
 ├── src/
|       ├── funcs.h / funcs.cpp   # чистые функции
|       └── array_ops.h / array_ops.cpp
|             # или str_ops / matrix_ops
└── tests/
        ├── test_funcs.cpp       # чистые функции
        └── test_variant.cpp     # семейство + алгоритм
```

```
cmake_minimum_required(VERSION 3.16)
project(lab_01) set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)


add_library(lab01_lib src/funcs.cpp
                      src/array_ops.cpp)
target_include_directories(lab01_lib PUBLIC src)

add_executable(lab01_main main.cpp)
target_link_libraries(lab01_main lab01_lib)

include(FetchContent)
FetchContent_Declare(googletest
    URL https://github.com/google/googletest/
          archive/refs/tags/v1.14.0.zip)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

enable_testing()
add_executable(lab01_tests tests/test_funcs.cpp
                           tests/test_variant.cpp) target_link_libraries(lab01_tests
GTest::gtest_main) include(GoogleTest)      gtest_discover_tests(lab01_tests)
```

Собрать и прогнать: cmake -B build→ cmake --build build→ ctest --test-dir build. Google Test подтягивается через **FetchContent**.

Google Test — ваши критерии качества

```
TEST(FuncsTest, FindMaxFillsValueByRef) {
    int data[] = {3, 9, 4, 1};
    int value = 0; EXPECT_TRUE(find_max(data, 4, value));
    EXPECT_EQ(value, 9);
}

TEST(FuncsTest, FindMaxEmptyReturnsFalse) {int value = 0;
    EXPECT_FALSE(find_max(nullptr, 0, value));
}

// test_variant.cpp (семейство A): создание/удаление,
// вставка в середину, удаление, бинарный поиск,
// затем алгоритм варианта на контрольном примере
```

- не менее **10 тестов**: чистые функции + операции семейства + алгоритм варианта
- каждый TEST— **свои данные**, не зависит от порядка запуска
- граничные случаи: пустой массив, один элемент, первая/последняя позиция

Поведение, а не код

тест проверяет, что **нашлось / не нашлось** и что записано в out-параметр.

ФИНАЛ

Что и когда сдавать

Срок

2 недели

с момента выдачи задания — по расписанию на паре.

Номер варианта

Назовите его при сдаче: он задаёт **семейство** структуры данных и **алгоритм**. Менять нельзя.

Что принести

Исходники, CMake-проект, тесты. Готовность к **демонстрации запуска** и короткому объяснению алгоритма.

Программа должна: **не падать** · **не течь** · корректно работать с **пустой структурой** · использовать **чистые функции** · проходить **свои тесты**.